FACULTY OF ARTS

KU LEUVEN

Natural Language Processing

Lecture 4: Neural Language Modeling

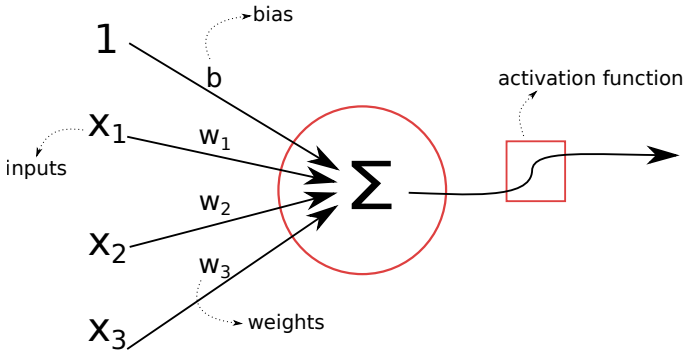
Tim Van de Cruys



Thursday, 15 October 2026

Reminder

- Neuron = logistic regression

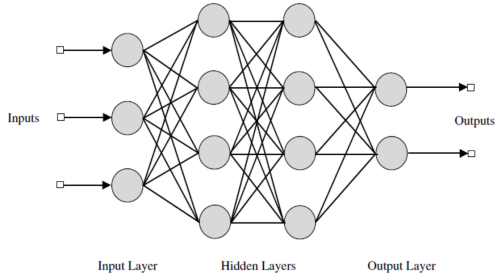


- We can feed an input vector to a bunch of logistic regression functions and get an output vector
- which can be fed to another layer of logistic regression functions

Feed-forward

Architecture

Multi-layer perceptron with 2 hidden layers



$$NN_{MLP2}(\mathbf{x}) = \mathbf{y} \quad (1)$$

$$\mathbf{h}^1 = g(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1) \quad (2)$$

$$\mathbf{h}^2 = g(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2) \quad (3)$$

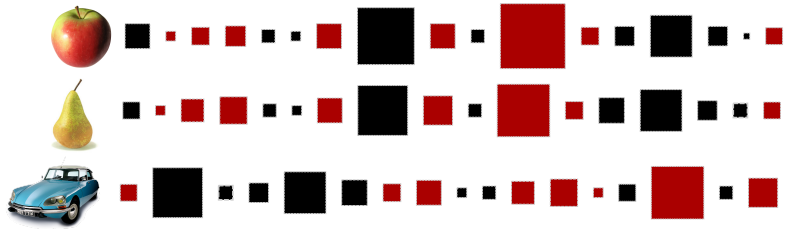
$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3 \quad (4)$$

$\mathbf{W}^1, \mathbf{W}^2, \mathbf{W}^3, \mathbf{b}^1, \mathbf{b}^2$ = parameters of the network (θ)

Use of multiple hidden layers: **deep learning**

Word embeddings

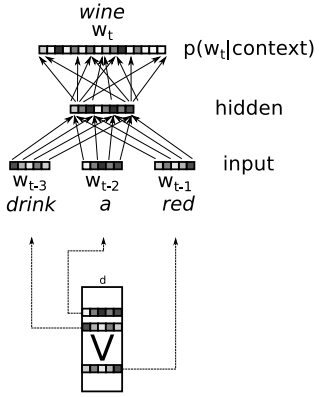
- Each word i is represented by a small, dense vector $\mathbf{v}_i \in \mathbb{R}^d$
- d is typically several thousand in current LLMs
- Matrix of size V (vocabulary size) $\times$ d (embedding size)
- words are 'embedded' in a real-valued, low-dimensional space
- Similar words have similar embeddings



Feed-forward network

Language modeling

- Predict the next word in a sequence, based on the previous word
- One non-linear hidden layer, one softmax layer for classification
- Choose parameters that optimize probability of correct word



Recurrent neural network

Model

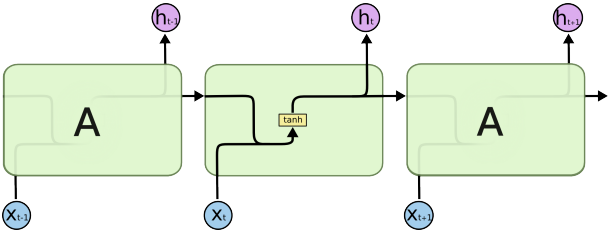
- $\mathbf{x}_1$: input layer (current word)
- $\mathbf{a}_1$: hidden layer of current timestep
- $\mathbf{a}_0$: hidden layer of previous timestep
- $\mathbf{U}$, $\mathbf{W}$ and $\mathbf{V}$: weights matrices
- $f(\cdot)$: element-wise activation function (e.g. tanh)
- $g(\cdot)$: softmax function to ensure probability distribution

$$\mathbf{a}_1 = f(\mathbf{U}\mathbf{x}_1 + \mathbf{W}\mathbf{a}_0) \tag{5}$$

$$\mathbf{y}_1 = g(\mathbf{V}\mathbf{a}_1) \tag{6}$$

Concrete RNN architectures

Simple RNN

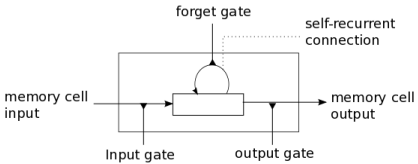


source images: `colah.github.io`

Concrete RNN architectures

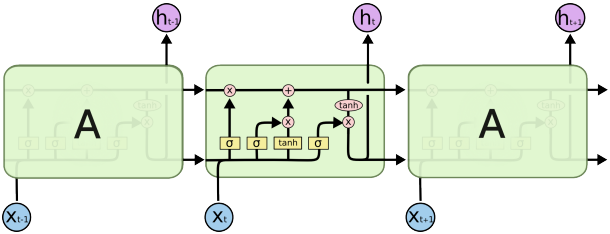
LSTM

- Long short term memory networks
- In practice, simple RNNs only able to remember narrow context (vanishing gradient)
- LSTM: complex architecture able to capture long-term dependencies



Concrete RNN architectures

LSTM



Transformer

- Basic building block of LLMs
- Like a fancy feedforward network, based on attention mechanism
- Transformer computes **contextualized** embeddings for words, based on surrounding words
- First, we'll explore a transformer model for **left to right** language modeling

`The astronomer watched the` **star**

`The audience watched the` **star**

Transformer

- Basic building block of LLMs
- Like a fancy feedforward network, based on attention mechanism
- Transformer computes **contextualized** embeddings for words, based on surrounding words
- First, we'll explore a transformer model for **left to right** language modeling

`The astronomer watched the` **`star`** `through the telescope.`

`The audience watched the` **`star`**

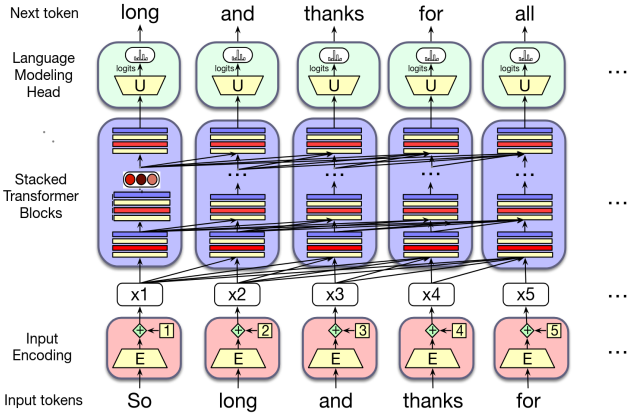
Transformer

- Basic building block of LLMs
- Like a fancy feedforward network, based on attention mechanism
- Transformer computes **contextualized** embeddings for words, based on surrounding words
- First, we'll explore a transformer model for **left to right** language modeling

`The astronomer watched the` **`star`** `through the telescope.`

`The audience watched the` **`star`** `in awe.`

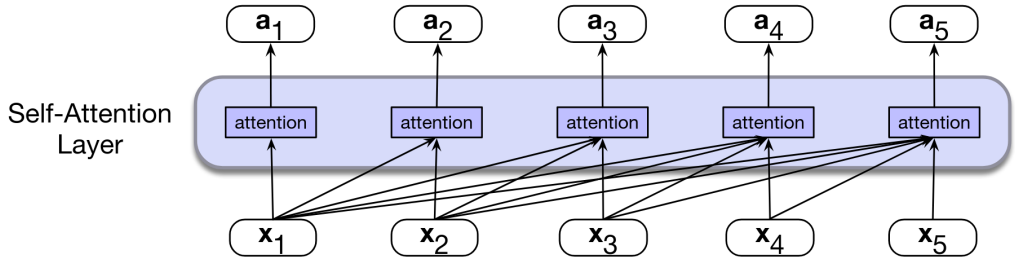
Transformer



Jurafsky and Martin (2026)

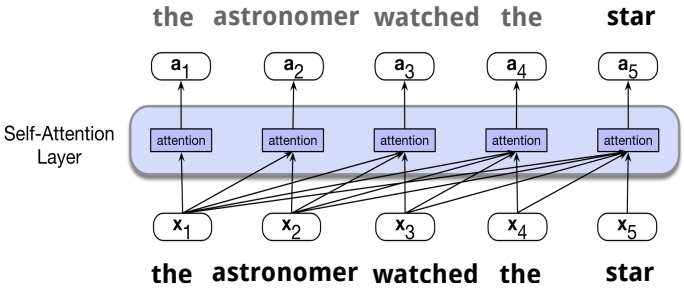
Attention

- Attention in causal language modeling (left to right)
- When processing $\mathbf{x}_i$, we attend to all inputs up to and including $\mathbf{x}_i$



Jurafsky and Martin (2026)

Attention: an intuitive simplification



$$score(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha_{ij} = softmax(score(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

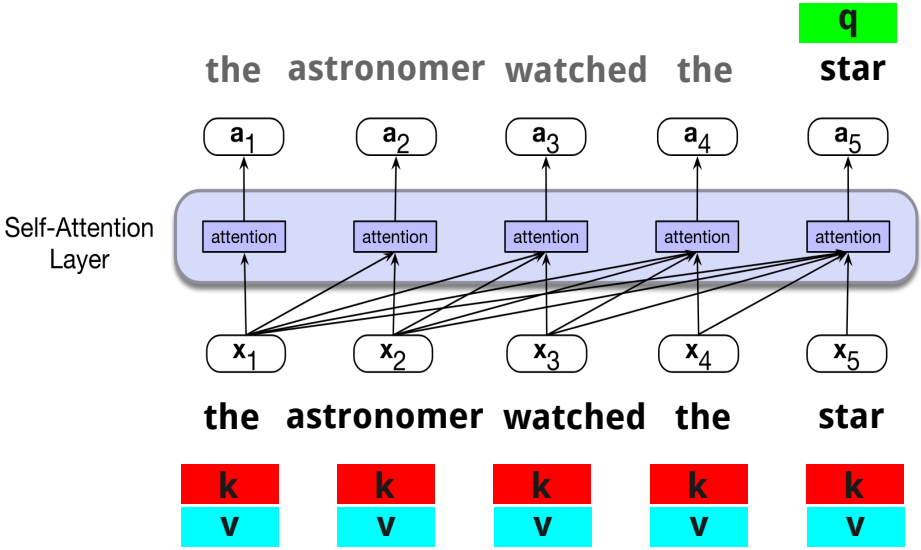
An attention head

- Instead of using original vectors ($\mathbf{x}_i$) directly, we create three derivative vectors for each $\mathbf{x}_i$
  - **query**: for current element, being compared to previous inputs ($\mathbf{q}_i$)
  - **key**: for preceding input, being compared to current element to compute similarity ($\mathbf{k}_i$)
  - **value**: value of preceding element that gets weighted and summed based on similarity ($\mathbf{v}_i$)
- Three matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$ to project original vector $\mathbf{x}_i$ into its query, key, value vector

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

- We'll use the dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$, and we'll sum over $\mathbf{v}_j$

An attention head



Attention computation

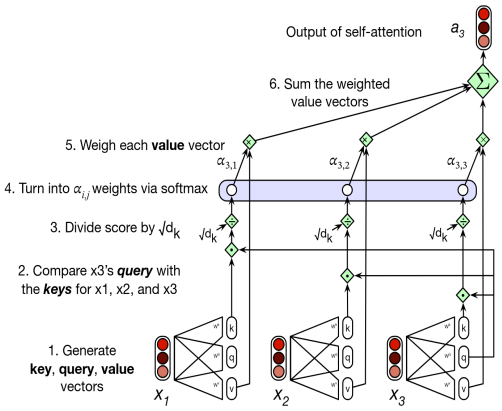
$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Visual representation



Jurafsky and Martin (2026)

Multi-head attention

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{Qc}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{Kc}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{Vc}; \quad \forall c, 1 \leq c \leq h$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

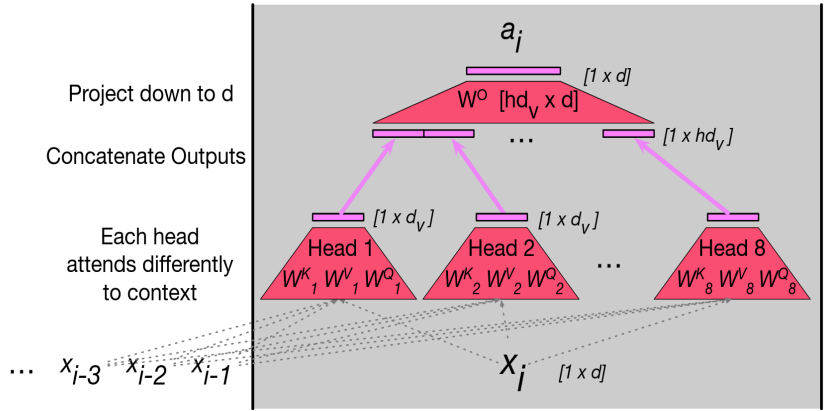
$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\text{head}_i^1 \oplus \text{head}_i^2 \cdots \oplus \text{head}_i^h)\mathbf{W}^O$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \cdots, \mathbf{x}_N]) = \mathbf{a}_i$$

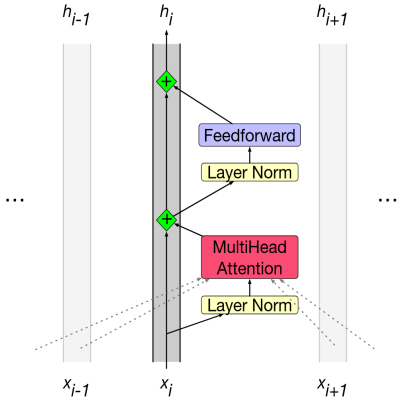
- Each head can focus on a specific purpose (linguistic relationships, contextual patterns)

Multi-head attention: visual representation



Jurafsky and Martin (2026)

The residual stream



Jurafsky and Martin (2026)

Layer Norm

- Normalization step applied to a single vector
- Variation of z-score in statistics: vector with mean 0 and standard deviation of 1

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

$$\sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sigma}$$

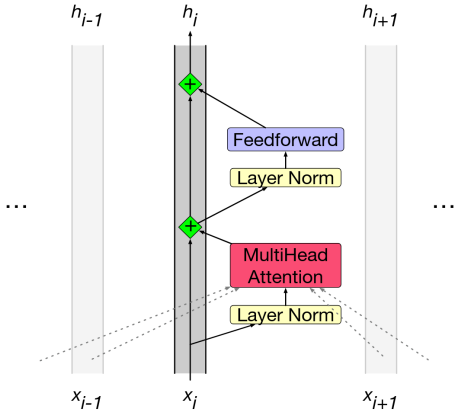
$$\text{LayerNorm}(\mathbf{x}) = \gamma\left(\frac{\mathbf{x} - \mu}{\sigma}\right) + \beta$$

Feedforward network

- Transformer block includes standard feedforward network (one hidden layer) in order to model non-linearities

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

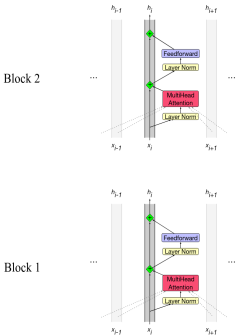
Putting it all together



$$\mathbf{t}_1^i = \text{LayerNorm}(\mathbf{x}_i)$$
$$\mathbf{t}_2^i = \text{MultiHeadAttention}(\mathbf{t}_1^i, [\mathbf{x}_1^1, \cdots, \mathbf{x}_N^1])$$
$$\mathbf{t}_3^i = \mathbf{t}_2^i + \mathbf{x}_i$$
$$\mathbf{t}_4^i = \text{LayerNorm}(\mathbf{t}_3^i)$$
$$\mathbf{t}_5^i = \text{FFN}(\mathbf{t}_4^i)$$
$$\mathbf{h}_i = \mathbf{t}_5^i + \mathbf{t}_3^i$$

Putting it all together

- Transformer blocks are stacked
- All vectors are same dimensionality *d*
- Note: computation can be parallelized for efficient computation



Putting it all together

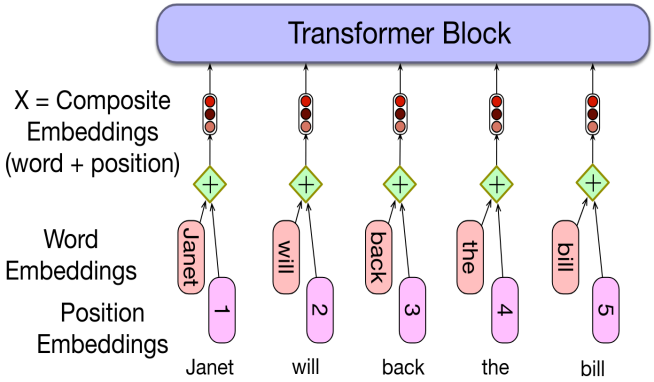
Current variants

- The architecture above is the original recipe; current LLMs (Llama, Qwen, Mistral, ...) differ in details
  - RMSNorm instead of LayerNorm: only rescaling, no re-centering
  - Gated feed-forward layers (SwiGLU) instead of ReLU
  - Rotary position embeddings (RoPE): relative position encoded in the attention computation, instead of absolute position embeddings
  - Grouped-query attention: several query heads share one key/value head, for faster inference
- The building blocks and the training objective are unchanged

Input: token embeddings plus position embeddings

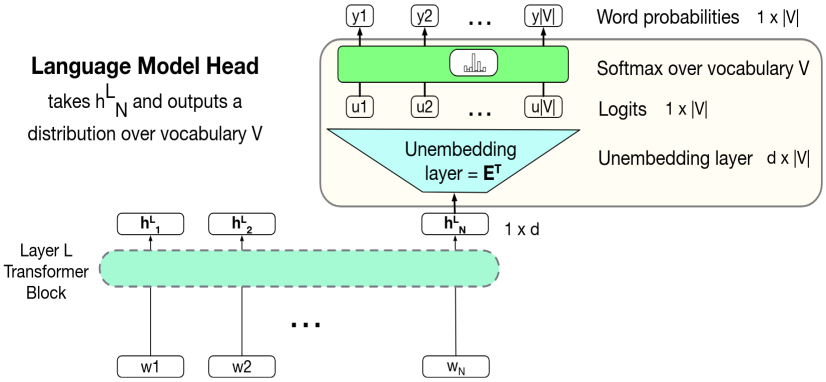
- Embedding matrix E has shape $[|V| \times d]$
  - One row for each of the $|V|$ tokens in the vocabulary
  - Each word is a row vector of d dimensions
- Position embeddings can be implemented in different ways, e.g. absolute position
  - Randomly initialized embeddings, one for each position (up to maximum length)
  - position embeddings (e.g. the embedding for position 17) are learned along with other parameters during training
- Input embedding is sum of token embedding and position embedding

Visual representation



Jurafsky and Martin (2026)

Language modeling head



Jurafsky and Martin (2026)

From logits to probabilities

- Logits are the unnormalized scores for each vocabulary token
- Softmax turns logits into probabilities

$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^T$$

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

Decoding

- The output of the model is not just one token, but a probability distribution over tokens
- How to pick the next token based on that distribution?
  - Most likely word: **greedy decoding**
  - Choose random words relative to the probability assigned by the model: **sampling**

E.g. random sampling:

$i \leftarrow 1$;
$w_i \sim p(w)$;
while $w_i \neq EOS$ **do**
  $i \leftarrow i + 1$;
  $w_i \sim p(w_i|w_{<i})$;
end

Improved decoding: restrict the vocabulary

- **Top-k sampling**
  - Only select the k most likely words, and normalize the restricted set of values to a proper probability distribution
- **Top-p sampling** (nucleus sampling)
  - Instead of choosing a fixed k, we restrict to the top p percent of the probability mass
  - Select smallest set of words such that

$$\sum_{w \in V(p)} P(w|\mathbf{w}_{<t}) \geq p$$

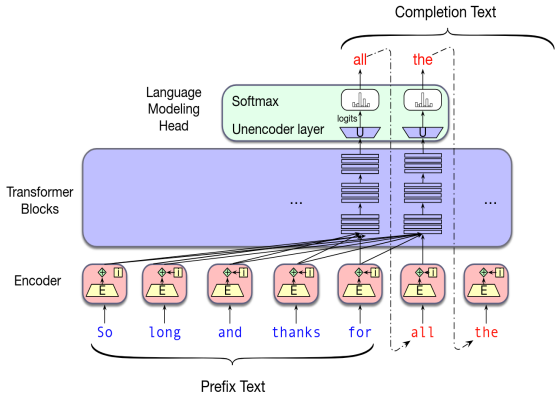
Improved decoding: reshape the probability distribution

- **Temperature sampling**
- Terminology from thermodynamics
  - high temperature system is flexible and can explore many states
  - low temperature system will explore subset of lower energy states
- Divide logit by temperature parameter τ before passing it through softmax: $\mathbf{y} = \text{softmax}(\mathbf{u}/\tau)$
- With lower τ, larger scores are passed to softmax
- Large inputs push high probability words higher and low probability words lower: distribution becomes more greedy

The power of scale

- LLMs: transformer architecture, applied on a large scale
- Lots of parameters, lots of data
- Just by predicting words, LLMs learn to solve many tasks
  - Sentiment analysis
  - Question answering
  - Summarization
  - ...

Conditional generation



Jurafsky and Martin (2026)

Examples

- p(w|*The sentiment of the sentence 'The new Bond movie is fantastic' is:*)
- p(w|*The capital of France is*)

Summarization

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For $89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

"We will not ship snow to any states in the northeast!" says Waring's website, ShipSnowYo.com. "We're in the business of expunging snow!"

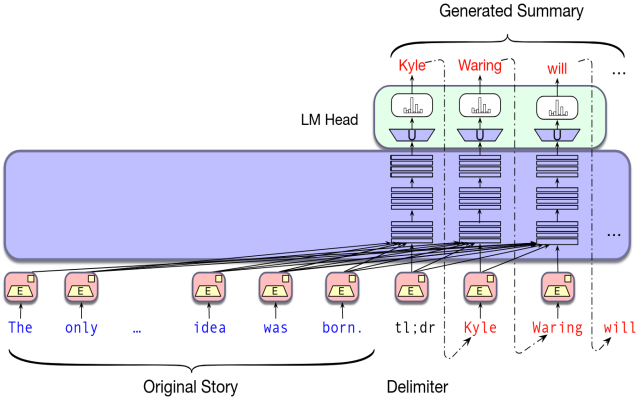
His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history.

According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born.

Summary:

Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

Summarization



Jurafsky and Martin (2026)

Pretraining with self-supervision

- First step in training LLM: pre-training on enormous amount of text
- Typically enormous amounts of text extracted from the web (e.g. CommonCrawl, and curated corpora derived from it such as FineWeb and Dolma): 10–15 trillion tokens for current models
- Training is 'self-supervised': no hand-labeled gold data, just next word as label

Objective function: cross-entropy loss

- Goal: assign high probability to true word w
- Incur high loss when model assigns low probability to w
- As a loss function, we make use of the negative log probability assigned to the true next word w
- Model weights are updated so that the loss is minimized

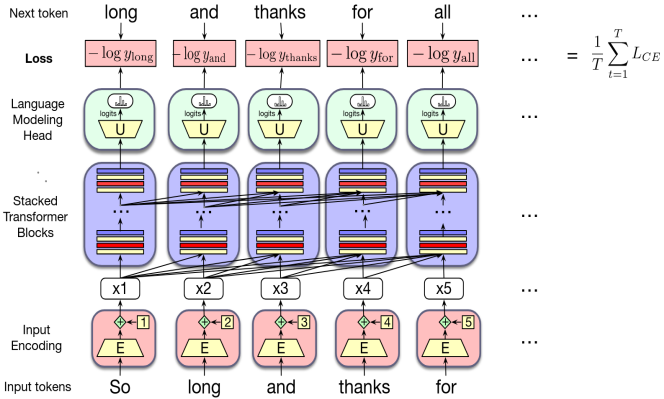
$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

Teacher forcing

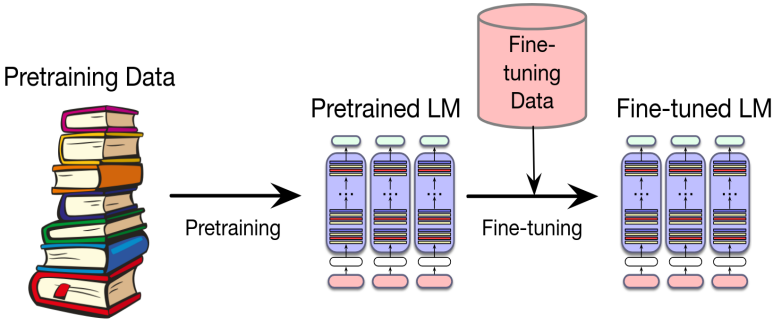
- At each token position t, the model sees the correct tokens $w_{1:t}$
- Computes the loss (negative log probability) for the next token w_{t+1}
- At the next token position $t+1$, we ignore what the model predicted for w_{t+1}
- Instead, we take the correct word w_{t+1}, add it to the context, and move on

Visual representation



Jurafsky and Martin (2026)

Finetuning



Jurafsky and Martin (2026)

Post-training

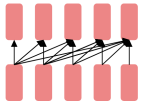
- A pretrained model predicts text; it does not follow instructions or converse. Turning it into an assistant takes further training steps
  - **Instruction tuning**: finetuning on instruction–response pairs (the loss is computed on the response tokens only)
  - **Preference learning**: human annotators compare pairs of responses; the preferences are used to train a reward model
  - **Alignment**: the model is optimized towards preferred responses, with reinforcement learning (RLHF, PPO) or directly (DPO)
- Finetuning large models is costly; parameter-efficient methods (e.g. LoRA) update only a small number of extra parameters

The downside of LLMs

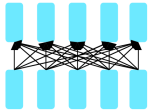
- Hallucination
- Copyright
- Privacy
- Bias/toxicity
- Misinformation
- Ecological impact

Three architectures for LLMs

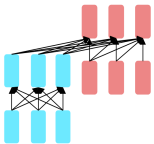
- Decoder
  - GPT, Llama, Qwen, ... (nearly all current LLMs)
  - Word prediction (and many tasks that can be framed as word prediction)



- Encoder
  - BERT: masked language modeling
  - Predict words based on left and right context
  - Usually fine-tuned with supervised data

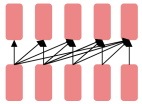


- Encoder-Decoder
  - T5
  - Map from one sequence to another
  - machine translation, speech recognition

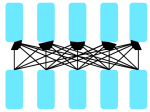


Three architectures for LLMs

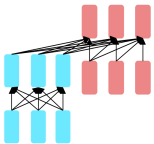
- Decoder
  - GPT, Llama, Qwen, ... (nearly all current LLMs)
  - Word prediction (and many tasks that can be framed as word prediction)



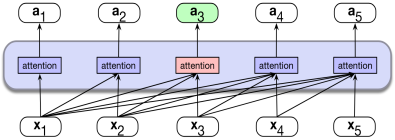
- **Encoder**
  - BERT: masked language modeling
  - Predict words based on left and right context
  - Usually fine-tuned with supervised data



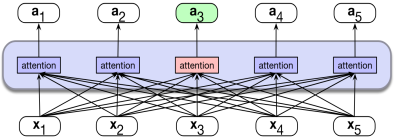
- Encoder-Decoder
  - T5
  - Map from one sequence to another
  - machine translation, speech recognition



Bi-directional attention



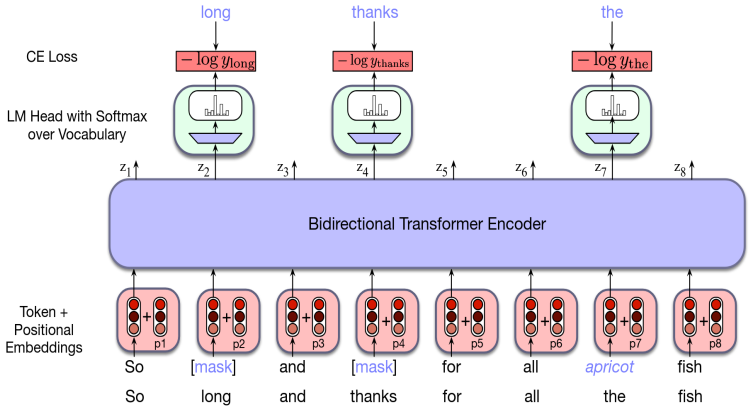
Causal self-attention layer



Bi-directional self-attention layer

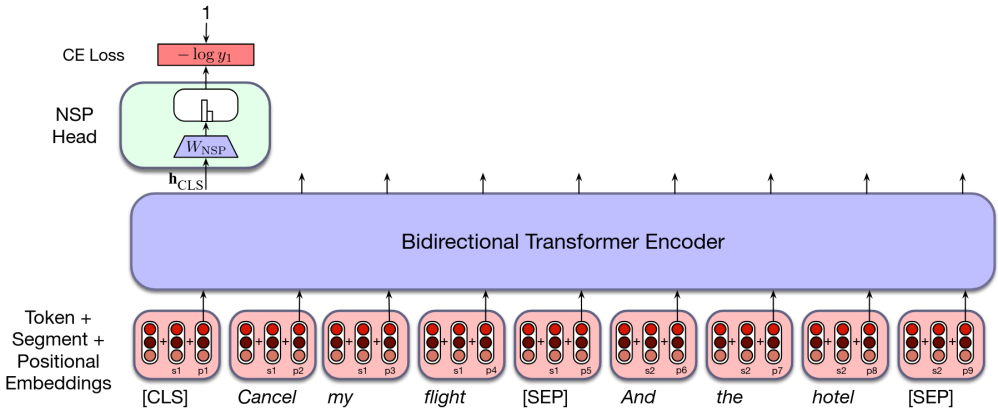
Jurafsky and Martin (2026)

Masked language modeling



Jurafsky and Martin (2026)

Next sentence prediction



Jurafsky and Martin (2026)

Contextualized embeddings

- BERT-like encoder models are great at capturing contextualized meaning



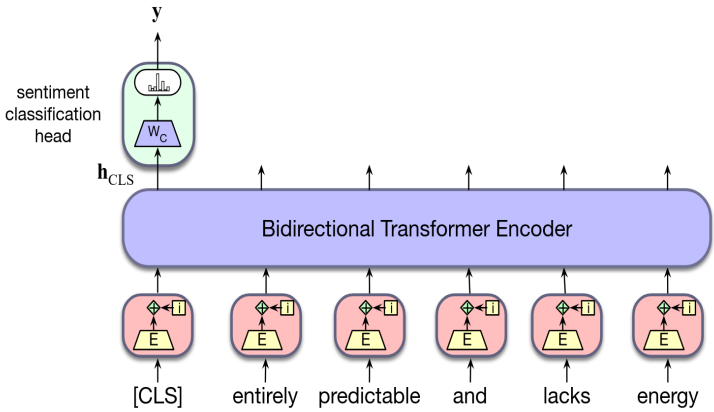
Coenen et al. (2019)

Fine-tuning for classification

- We take our pre-trained encoder model (trained on a large amount of text using masked language modeling)
- We fine-tune a classifier on a curated set of hand-labeled training data, using an appropriate **head** on top of the pretrained model
- Pre-trained parameters can be frozen or (minimally) adjusted

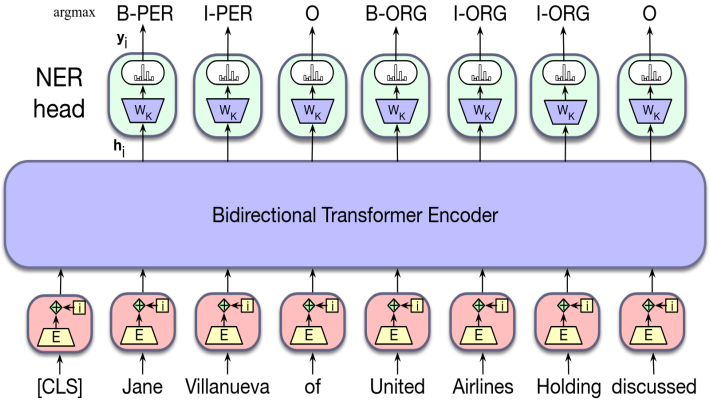
$$\mathbf{y} = \text{softmax}(\mathbf{h}^L_{CLS}\mathbf{W}_C)$$

Sequence classification



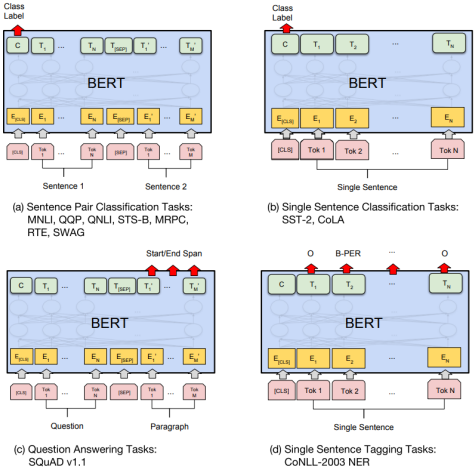
Jurafsky and Martin (2026)

Sequence labeling



Jurafsky and Martin (2026)

Different classification configurations



Devlin et al. (2019)